

Compilación y ensamblado

- Apunte del Instituto Tecnológico de Saltillo (México) - Carrera: Ingeniería en sistemas computacionales. Asignatura: Lenguajes de interfaz.
- Principles of Computer Architecture - Miles J. Murdocca, Vincent Heuring

5 pasos para crear un programa

Para la creación de un programa es necesario seguir, por lo menos, cinco pasos:

- Diseño del algoritmo
- Codificación del mismo
- Traducción a lenguaje máquina
- Prueba del programa
- Depuración

En la etapa de diseño se plantea el problema a resolver y se propone la mejor solución, creando diagramas esquemáticos o pseudocódigo utilizados para el mejor planteamiento de la solución. Se debe recordar que si la lógica o el algoritmo del programa está mal, nunca va a funcionar por más cambios en el código que se hagan. La codificación del programa consiste en escribir en un editor de texto o en un IDE (Entorno de Desarrollo Integrado) el programa, en algún lenguaje de programación (en este caso en ensamblador 8086), tomando como base la solución propuesta en el paso anterior. La traducción al lenguaje de máquina es la creación del programa objeto, esto es, el programa escrito como una secuencia de ceros y unos que pueda ser interpretado por el procesador. La prueba del programa consiste en verificar que el programa funcione sin errores, o sea, que haga lo que tiene que hacer. La última etapa es la eliminación de las fallas detectadas en el programa durante la fase de prueba. La corrección de una falla normalmente requiere la repetición de los pasos comenzando desde el primero o el segundo. Para crear un programa en ensamblador se puede utilizar el debugger, que se encuentra en cualquier PC con el sistema operativo MS-DOS o simularlo, lo cual lo pone al alcance de cualquier usuario que tenga acceso a una máquina con estas características. Debug solo puede crear archivos con extensión .EXE, y por las características de este tipo de programas no pueden ser mayores de 64 kb, además deben comenzar en el desplazamiento, offset, o dirección de memoria 0100h dentro del segmento específico.

Lenguaje de alto nivel y lenguaje de bajo nivel



Lenguajes de Alto Nivel:

codifican comandos poderosos.
cada uno de los cuales puede generar muchas instrucciones en lenguaje de máquina.

Un lenguaje de alto nivel utiliza un *compilador* para traducir el código fuente a lenguaje de máquina (técnicamente, código objeto).



Lenguajes de Bajo Nivel:

lenguaje ensamblador de bajo nivel codifican instrucciones simbólicas,
cada una de las cuales genera una instrucción en lenguaje de máquina.

Un lenguaje de bajo nivel utiliza un *ensamblador* para realizar la traducción.

Un programa *enlazador* para ambos niveles, alto y bajo ; completa el proceso al convertir el código objeto en lenguaje ejecutable de máquina.

- Proporciona más control sobre el manejo particular de los requerimientos de hardware.
- Genera módulos ejecutables más pequeños y más compactos.
- Con mayor probabilidad tiene una ejecución más rápida.

Una práctica común es combinar los beneficios de ambos niveles de programación:
codificar el grueso de un proyecto en un lenguaje de alto nivel y los módulos críticos (aquellos que provocan notables retardos) en lenguaje ensamblador.

Compilado y enlace

Un programa escrito en un lenguaje de programación, no puede ser ejecutado directamente por un ordenador, sino que debe ser traducido a lenguaje máquina.

Las etapas por las que debe pasar un programa escrito en un lenguaje de programación, hasta poder ser ejecutable son:

Compilado → Pasa del programa fuente escrito en alto nivel a código objeto.

Linkeado → Pasa de código objeto a ejecutable (.exe o .com)

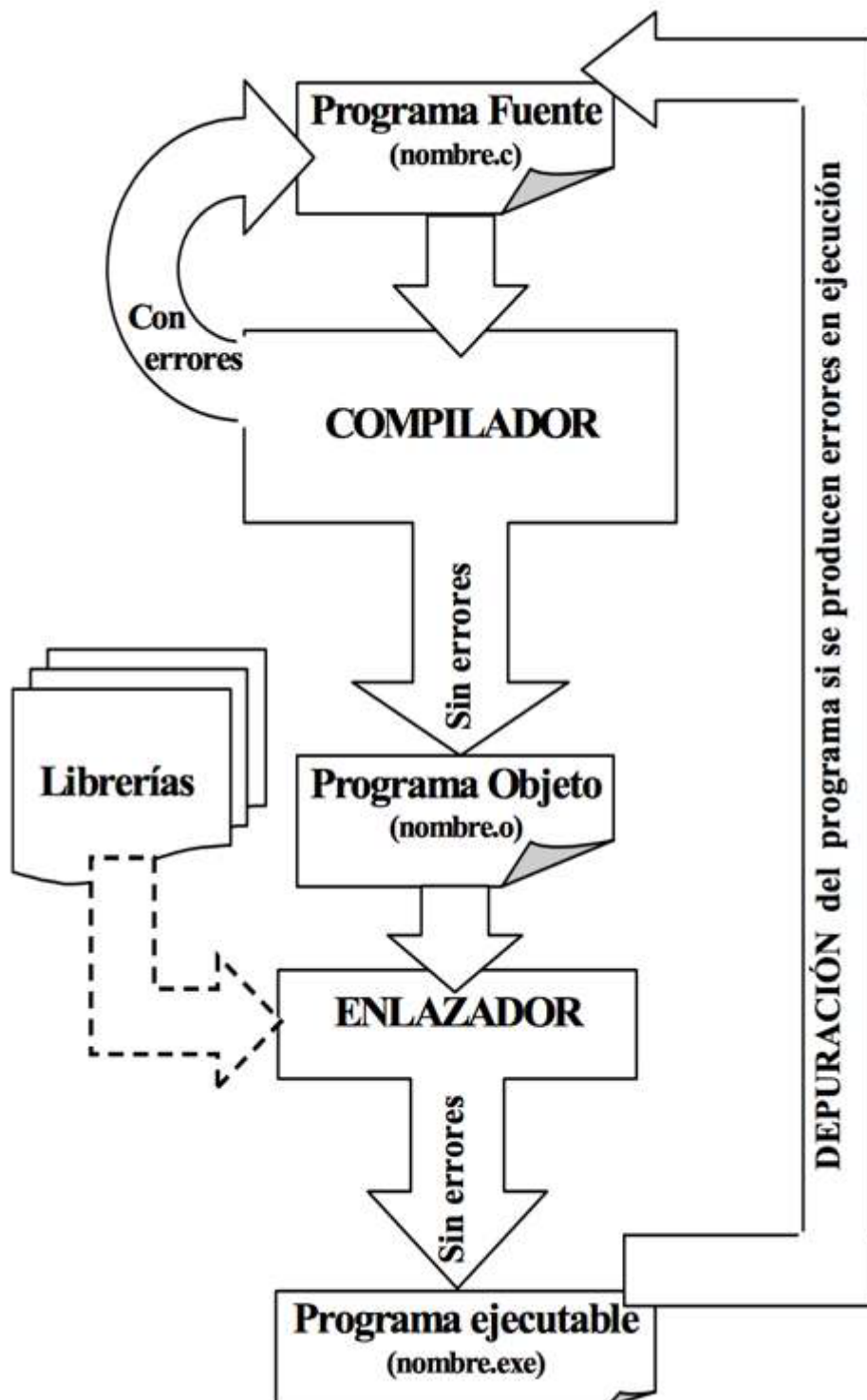
Programa fuente: Programa escrito en un lenguaje de alto nivel (texto ordinario que contiene las sentencias del programa en un lenguaje de programación). Necesita ser traducido a código máquina para poder ser ejecutado.

Compilador: Programa encargado de traducir los programas fuentes escritos en un lenguaje de alto nivel a lenguaje máquina y de comprobar que las llamadas a las funciones de librería se realizan correctamente.

Usaremos el GCC (GNU Compiler Collection) descargado de <https://www.mingw-w64.org/downloads/>

Programa (o código) objeto: Es el programa fuente traducido (por el compilador) a código de máquina. Aún no es directamente ejecutable. El contenido de un archivo objeto es independiente del lenguaje en que fue escrito el archivo fuente.

Al generarse los programas objetos y los archivos objeto, también se crean las Tablas de Símbolos que luego se utilizan durante el proceso de enlace o linking para resolver todas las referencias pendientes.



Enlazador o linker: Los módulos .OBJ son archivos intermedios no ejecutables que pueden ser desarrollados por diferentes programadores en forma separada, en diferentes módulos. El linker transforma los módulos objeto en programas ejecutables. Luego del enlace, los módulos objeto forman un único módulo de carga.

Programa Ejecutable: Traducción completa a código máquina, realizada por el enlazador, del programa fuente y que ya es directamente ejecutable.

Compilación

El proceso de traducir un programa escrito en lenguaje ensamblador (simbólico o mnemónico) al lenguaje de máquina es muy directo, debido a que hay una relación uno a uno entre las sentencias del lenguaje simbólico y los códigos de máquina equivalentes. Los lenguajes de alto nivel, por el contrario, presentan un problema mucho más complejo.

Si se considera una simple sentencia de asignación en lenguaje de alto nivel, por ejemplo:

$$A = A + 4$$

La conversión de esta sentencia en una o más sentencias del lenguaje simbólico enfrenta al compilador con una cantidad de tareas relativamente complejas:

- Reconocer dentro del texto del programa los símbolos básicos del lenguaje, por ejemplo, los identificadores tales como A y B, definiciones tales como la del valor constante 4 y del indicadores como = y +. Esta parte de la compilación se suele conocer como **análisis lexicográfico**.
- Analizar los símbolos para reconocer la estructura de la programación subyacente. En el ejemplo, al analizar (parsear) se debe reconocer que la sentencia utilizada corresponde a una sentencia de asignación del tipo:

Identificación "=" Expresión

En la que, posteriormente se analizará Expresión con el objeto de detectar una forma del tipo:

Identificador "+" Constante

Este procedimiento se denomina **análisis sintáctico**.

- **Análisis de nombres:** asociar los nombres A y B con variables particulares del programa y, posteriormente, con posiciones particulares de memoria en las que se almacenarán dichas variables al momento de la ejecución del programa.
- **Análisis de tipo:** determinar el tipo de todos los datos requeridos. En el ejemplo anterior, las variables A y B y la constante 4 se reconocerían, en ciertos lenguajes, como de tipo entero. Los análisis de nombre y de tipo se relacionan a menudo para incluirlos bajo el nombre de **análisis semántico**.
- **Asignación de acciones y generación de código:** es la acción de asociar las sentencias de programa con la secuencia apropiada del lenguaje ensamblador. En el ejemplo de la sentencia anterior ($A = A + 4$), la secuencia de lenguaje simbólico podría, por ejemplo, ser la siguiente:

```
mov AL, [200] ; valor de A
mov AH,[201] ; el número 4
add AL, AH   ; la suma
mov [200], AL ; redefinir el valor de A
```

Además de las mencionadas, existen acciones adicionales que deben ser resueltas por el compilador, como la asignación de variables a registros, el control de uso de registros y, si así lo quisiera el programador, la optimización del programa.

Cuando se desarrolla el compilador, se debe incluir en su estructura la información acerca de la arquitectura de programación particular del procesador para el cual se desarrolla.

La arquitectura de programación de la plataforma sobre la que el compilador se ejecuta no necesita ser igual a la del código que el compilador genera, proceso conocido como compilación cruzada o cross compilation.

El proceso de ensamblado

El proceso de traducción de un programa expresado en lenguaje simbólico se conoce como proceso de ensamblado. Este proceso es lineal y relativamente simple, dado que hay una relación directa y unívoca entre las sentencias del lenguaje simbólico y su correspondientes expresiones en lenguaje de máquina.

Esto marca una diferencia con el proceso de compilación, por ejemplo, en el que una sentencia de lenguaje de alto nivel dado puede traducirse como una buena cantidad de sentencias de lenguaje de máquina.

Si bien el ensamblado es un proceso lineal, es tedioso y proclive al error si se hace a mano. De hecho, el programa ensamblador es una de las primeras herramientas de software desarrolladas luego de la invención de la computadora electrónica digital.

Los ensambladores comerciales deben ofrecer al menos las siguientes prestaciones:

- Permitir que el programador especifique la ubicación de las variables y programas al momento de ejecución. Sin embargo, en la mayoría de los casos el programador no especifica una dirección absoluta de comienzo de un programa, dado que el programa puede ser movido o reubicado por el programa vinculador o linker y, probablemente, por el programa de carga o loader.
- Ofrecerle al programador la posibilidad de inicializar los valores de los datos en memoria antes de la ejecución del programa.
- Proveer expresiones mnemotécnicas en el lenguaje de programación para todas las instrucciones del lenguaje de máquina y modos de direccionamiento, y traducir las sentencias válidas del lenguaje simbólico hacia los valores binarios equivalentes en el lenguaje absoluto.
- Permitir el uso de rótulos simbólicos para definir o representar direcciones y constantes.
- Ofrecer al programador una forma de especificar la dirección de comienzo de un programa, si existiera. Por ejemplo, no habría dirección de comienzo si el módulo que se traduce es una función o un procedimiento.
- Ofrecer cierto grado de cálculo al momento del ensamblado.
- Incluir un mecanismo que permita la definición de variables en un programa escrito en lenguaje simbólico y el uso de las mismas en otro programa ensamblado por separado.
- Proveer la expansión de macro y rutinas, o sea, rutinas que puedan definirse una vez y utilizarse tantas veces como sea necesario.

Ensambladores de dos pasadas

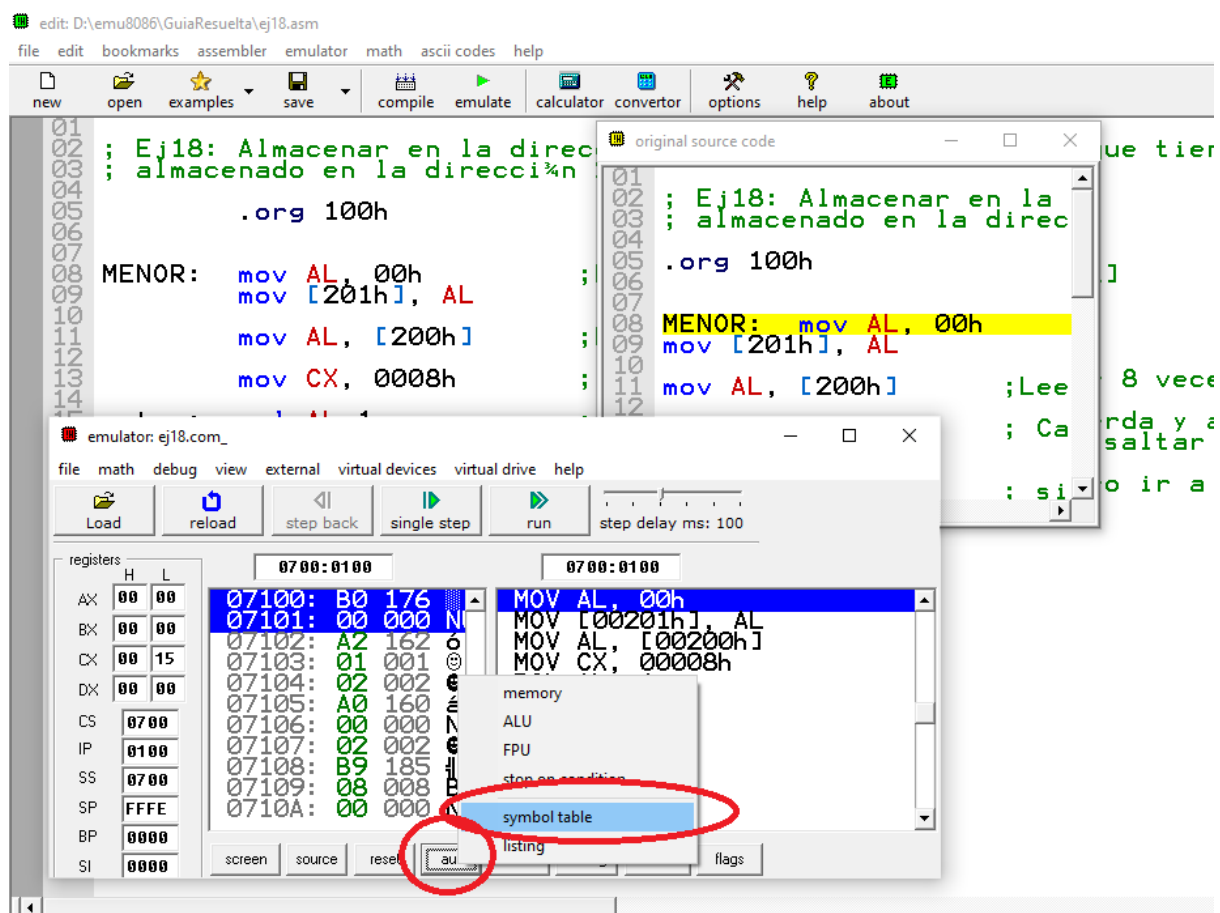
La mayoría de los ensambladores recorren dos veces el texto escrito en lenguaje simbólico, por lo que se los conoce como ensambladores de dos pasadas.

En la primera pasada el ensamblador se dedica a:

- Determinar las direcciones de todos los datos.
- Determinar las direcciones de todas las instrucciones.
- Seleccionar una instrucción de lenguaje de máquina para cada instrucción del lenguaje simbólico.
- Operaciones aritméticas de cálculo de direcciones.
- Guardar rótulos y constantes en la tabla de símbolos.

Las direcciones de los datos y las instrucciones se determinan en el momento del ensamblado mediante un contador similar al contador de programa, al que se conoce como contador de posición o location counter.

A medida que se desarrolla el proceso de ensamblado, este contador de posición lleva el control de las direcciones de la instrucción o del elemento de datos corriente. Generalmente se lo inicializa en cero al comienzo de la primer pasada y se lo incrementa en pasos equivalentes al tamaño de cada instrucción. La directiva `.org` hace que el contador se ubique en el valor especificado por la misma. Por ejemplo, en el caso de encontrar la sentencia `.org 1000` el contador de posición adoptará el valor 1000, por lo que la instrucción o datos siguientes se asignará a dicha dirección. Durante esa pasada, el ensamblador realiza también cualquier operación aritmética necesaria e inserta las definiciones de todos los rótulos y constantes en una tabla de símbolos.



La razón principal para requerir una segunda pasada tiene que ver con la necesidad de permitir el uso de símbolos dentro de un programa con anterioridad a que sean definidos, lo que se suele conocer como referencia previa. Luego de la primera pasada, el ensamblador tendrá todos los símbolos definidos e ingresados en la tabla de símbolos, por lo que durante la segunda pasada podrá generar el código de máquina, insertando en el mismo los valores de los símbolos, ya conocidos para ese momento.

En la primera pasada de un proceso de ensamblado de dos pasadas se crea una tabla de símbolos. Un símbolo puede ser un rótulo o un nombre simbólico que se refiera a un valor utilizado durante el proceso de ensamblado.

Luego de creada la tabla de símbolos comienza la segunda vuelta de ensamblado. El programa se lee por segunda vez, empezando desde la sentencia `.begin` para generar en esta vuelta el código objeto.

Tareas finales del programa ensamblador

Luego de completar el proceso de traducción, el ensamblador debe agregar al módulo traducido información adicional para uso de los programas de enlace y carga (linker y loader):

- El nombre y tamaño del módulo.
- La dirección del símbolo de comienzo, por ejemplo, en C se especifica el inicio con la función llamada `main`.
- Información acerca de símbolos globales y externos. El programa de enlace necesitará conocer la dirección de cualquier símbolo global definido en el módulo y exportado por el mismo, así como necesitará saber qué símbolo se encuentra indefinido en el módulo porque se definió como global en otro módulo.
- Información acerca de las rutinas de biblioteca.
- Los valores de cualquier constante que deba cargarse en la memoria.
- Información de reubicación: cuando se invoca el programa de enlace, la mayoría de los módulos a vincular deben reubicarse a medida que se los concatena. Todo el procedimiento de reubicación de módulos es complicado debido a que algunas referencias a direcciones pueden reubicarse y otras no.

Ubicación del programa en memoria

Los programas se encuentran localizados en memoria en una dirección especificada por la directiva `.org`. Es responsabilidad del ensamblador indicar qué símbolos son reubicables. La característica de reubicable o no que tiene un símbolo determinado depende tanto del lenguaje simbólico como de las convenciones del sistema operativo. En cualquier caso, la información de reubicación se incluye en un diccionario de reubicación en el módulo de ensamblado, para que sea utilizado por el editor de enlace o linker y por el cargador o loader.

Enlace y carga

La mayoría de las aplicaciones de cualquier tamaño tiene una cantidad de módulos que han sido compilados o ensamblados por separado. Estos módulos pueden haber sido generados por distintos lenguajes de programación o pueden integrar una biblioteca provista como parte del ambiente del lenguaje de programación o del sistema operativo.

Un editor de enlace o linker, es un programa que combina programas ensamblados por separados llamados módulos objeto en un único programa, o módulo de carga. El linker resuelve todas las referencias globales y externas y reubica las direcciones de los diferentes módulos. El módulo de carga puede ser cargado en memoria por medio de un cargador o loader, que también puede necesitar modificar direcciones si el programa se carga en una dirección distinta a la dirección de origen de carga usada por el linker.

Una técnica relativamente nueva, denominada bibliotecas de enlace dinámico (dll dynamic links libraries), popularizada por Microsoft en su sistema operativo Windows y presente en

formas similares en otros sistemas operativos, pospone el enlace de algunos componentes hasta que sean requeridos efectivamente en el momento de la ejecución.

Enlace (linking)

Al combinar los módulos ensamblados o compilados por separado en un módulo de carga único el programa de enlace debe:

- Resolver referencias de direcciones externas a los módulos a medida que los vincula.
- Reubicar cada módulo por medio de la combinación más apropiada de los mismos. Durante este proceso de reubicación muchas de las direcciones del módulo pueden llegar a modificarse para reflejar su nueva ubicación.
- Especificar el símbolo de comienzo del módulo de carga.
- Si el módulo de memoria incluye más de un segmento de memoria, debe especificar los contenidos e identidades de esos diversos segmentos.

Referencias externas

Al resolver las referencias de direcciones, el programa de enlace necesita distinguir los nombres de los símbolos locales (aquellos que se utilizan dentro de un único módulo fuente) de los nombres de los símbolos globales (los que se usan en más de un módulo). Esto se logra mediante el uso, durante el proceso de ensamblado, de las directivas `.global` y `.extern`.

La directiva `.global` le indica al ensamblador que debe señalar el símbolo como disponible para otros módulos objetos durante la fase de enlace.

La directiva `.extern` identifica a un rótulo usado en un módulo pero que se encuentra definido en otro.

Por lo tanto, se utiliza la directiva `.global` en el módulo en el que se define ese símbolo y se utiliza la directiva `.extern` en cada uno de los módulos que tenga referencia a aquel. Nótese que solo los rótulos de direcciones pueden ser globales o externos.

Reubicación

La idea es con programa ensamblado en una dirección inicial como 2048 pueda cargarse no a partir de ella sino a partir de otra dirección, como por ejemplo, 3000 para permitir esta modificación todas las referencias a direcciones reubicables incluidas en el programa deben incrementarse en $3000 - 2048 = 952$

La reubicación se realiza a través del programa linker, de modo tal que las direcciones reubicables se modifican en el mismo valor en que se modificó la dirección de origen de carga. En cambio, las direcciones absolutas, no reubicables, tales como la dirección más alta posible para la pila, se mantienen sin modificación independientemente del origen de carga.

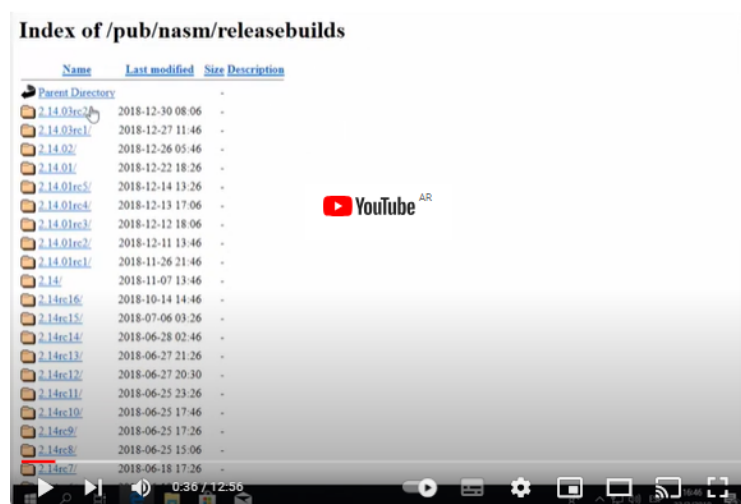
Carga

El cargador o loader es un programa que ubica el módulo de carga en la memoria principal. Conceptualmente, las tareas del programa cargador no son complejas. Debe cargar los diversos segmentos de memoria con valores apropiados e inicializar ciertos registros, como el puntero de pila y el contador de programa, a sus valores iniciales.

El programa de enlace debe combinar varios módulos objetos en un único módulo de carga, mientras el cargador simplemente modifica las direcciones reubicables que encuentre dentro de un módulo de carga dado, para permitir la coexistencia en memoria de varios programas en forma simultánea.

Instalación del compilador

Descargar e instalar el compilador “GNU Compiler Collection” (GCC) de acuerdo con las instrucciones del siguiente video. Saltar la instalación del ensamblador NASM dado que usamos EMU8086 y no vamos a usar NASM:



<https://youtu.be/sKt2Pb8ZmcY>



Compilar un programa

Para compilar un programa con el compilador GNU Compiler Collection" (GCC) es conveniente crear una carpeta donde se van a guardar, por ejemplo, D:\FIUBA\9557\U4\PROGRAMAS\

Escribir un archivo en C con cualquier editor o IDE y guardar el código fuente en la carpeta recién generada:



Nombre	Ext	Tamaño	Fecha	Atrib.
[..]		<DIR>	04/11/2023 10:03	----
wile_float	c	245	04/11/2023 09:15	-a--

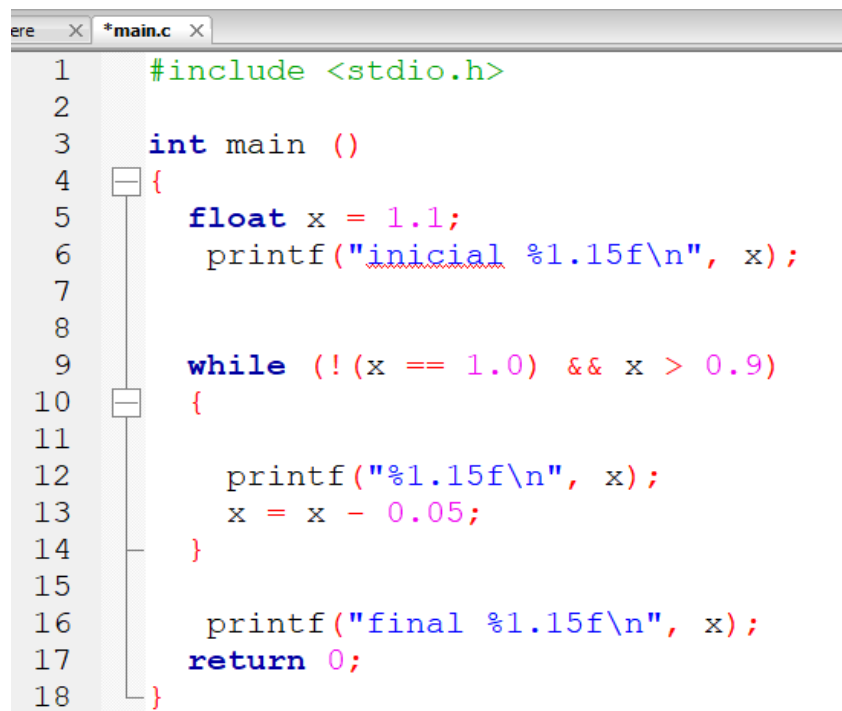
A modo de prueba, se puede usar el siguiente programa que muestra que en punto flotante (IEEE754) pueden haber valores que nunca se alcancen, por lo que en ocasiones conviene comparar con mayor o menor en vez de usar igual.

```
#include <stdio.h>
```

```
int main ()
{
    float x = 1.1;
    printf("inicial %1.15f\n", x);

    while (!(x == 1.0) && x > 0.9)
    {
        printf("%1.15f\n", x);
        x = x - 0.05;
    }

    printf("final %1.15f\n", x);
    return 0;
}
```



```
1  #include <stdio.h>
2
3  int main ()
4  {
5      float x = 1.1;
6      printf("inicial %1.15f\n", x);
7
8
9      while (!(x == 1.0) && x > 0.9)
10     {
11
12         printf("%1.15f\n", x);
13         x = x - 0.05;
14     }
15
16     printf("final %1.15f\n", x);
17     return 0;
18 }
```

Se puede ejecutar para verificar que el programa está bien hecho:

```
"D:\UBA\6670\6670_Evaluaciones\Parcial 1\float_while\main.exe"
inicial 1.100000023841858
1.100000023841858
1.050000071525574
1.000000119209290
0.950000107288361
0.900000095367432
final 0.850000083446503

Process returned 0 (0x0)   execution time : 0.062 s
Press any key to continue.
```

Para compilar el programa hay que abrir la consola de comandos CMD dentro de la misma carpeta en la que se encuentra el programa fuente:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.19045.3570]
(c) Microsoft Corporation. Todos los derechos reservados.

d:\FIUBA\9557\U4\PROGRAMAS>
```

Con el comando "dir" de DOS se puede comprobar que el programa fuente se encuentra dentro de la carpeta:

```
Directorio de d:\FIUBA\9557\U4\PROGRAMAS

04/11/2023  10:03    <DIR>          .
04/11/2023  10:03    <DIR>          ..
04/11/2023  09:15                245 wile_float.c
               1 archivos                245 bytes
               2 dirs   9.261.654.016 bytes libres
```

En el prompt escribir:

gcc, un espacio, el nombre del programa fuente con su extensión, otro espacio, el comando -o, otro espacio y el nombre del programa ejecutable sin la extensión:

```
d:\FIUBA\9557\U4\PROGRAMAS>gcc wile_float.c -o programa_de_prueba
```

Después de pulsar enter se generará el programa ejecutable:

d:\FIUBA\9557\U4\PROGRAMAS*.*				
Nombre	Ext	Tamaño	Fecha	Atrib.
[..]		<DIR>	04/11/2023 10:31	----
programa_de_prueba	exe	54.024	04/11/2023 10:31	-a--
wile_float	c	245	04/11/2023 09:15	-a--

Para ejecutarlo, lo hacemos desde la consola:

```
d:\FIUBA\9557\U4\PROGRAMAS>programa_de_prueba
inicial 1.100000023841858
1.100000023841858
1.050000071525574
1.000000119209290
0.950000107288361
0.9000000095367432
final 0.850000083446503
```

- Para generar el archivo objeto se usa el comando:
`gcc -c -g while_float.c`
- Para generar el programa en assembler se usa el comando
`gcc -S while_float.c`
- Para ver juntos el código en C y el código assembler generado se usa el comando
`gcc -c -g -Wa,-a,-ad while_float.c > while_float.lst`

Para compilar un programa fuente en C++, cuya extensión es .cpp, la única diferencia es que en vez de usar el comando gcc, se usa g++:

g++, un espacio, el nombre del programa fuente con su extensión, otro espacio, el comando -o, otro espacio y el nombre del programa ejecutable sin la extensión.

Más información en: <https://gcc.gnu.org/>